Learning to rank

Vladislav Goncharenko

Slides by Ilya Boytsov

girafe ai × spring 2025

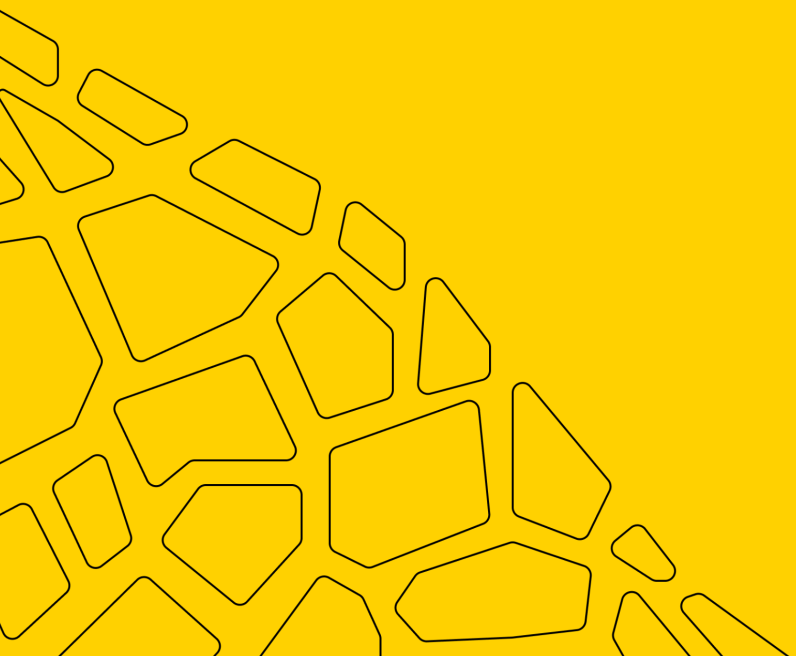
Recap

Lecture 4:
Sequential Recommenders

1. Обучаемые эмбединги
2. YoutubeDNN
3. Deep Interest Network
4. Behaviour Sequence Transformer
5. SASRec
6. BERT4Rec
7. PinnerSage
8. PinnerFormer

Outline

Lecture 3:
Metrics and losses for ranking



1. Ranking problem statement
2. Offline metrics
    a. Special properties
3. Online metrics
4. Losses
    a. RankNet
    b. LambdaRank
    c. LambdaMART
    d. YetiRank
5. Implementations

Views on recommendation problem

1. Interaction matrix restoration
   a. Bounded to collaborative information only
   b. Holistic view on dataset
   c. Usually solved as regression
2. Items ranking
   a. Features can be anything
   b. Each group of objects treated separately (e.g. each user)
   c. Requires different problem statement

Ranking problem statement

Given N queries (users in RecSys) Q = {q1, ..., qN}.

Each query i has K (1-1000) documents (items) D = {di_1, ..., di_K}
and their relevance levels R = {r_i1, ..., r_iK}

Each query and document has feature vectors of E and M.

So one point of data (q, d, r)

Our goal is to build a model that restores **the sequence** of documents for each query according to relevance R.

We look for a model f(q_i, d_j) = t_ij. (t_i1, ... t_iK) ~ (r_i1, ... , r_iK). t_i1 != r_i1

We are not needend to predict R values.

Ranking problem statement

Дано N запросов (queries) Q = {q1, ..., qN}.

Каждый запрос состоит из K документов D = {d1, ..., dK}
и их меток релевантности L = {l1, ..., lK}

Каждый запрос qi описывается вектором признаков размерности E,
а документ di - размерности M.

Тогда цель обучения - это построить такую модель, которая наилучшим образом будет восстанавливать порядок документов в соответствии с истинными метками релевантности L.

f(q, d) = score_{qd}

Ranking problem statement

BMW X3 20i xDrive II (F25) — 2012

1 685 000 ₽

от 34 900 ₽ / мес.

2.0 л / 184 л.с. / Бензин — Полный

Автомат — Серый

Внедорожник 5 дв.

Безопасная сделка | Отчёт ПроАвто

Рязань (180 км от Москвы), 40 минут назад

BMW X3 20d xDrive III (G01) — 2018

3 790 000 ₽

от 62 150 ₽ / мес.

2.0 л / 190 л.с. / Дизель — Полный

Автомат — Белый

Внедорожник 5 дв.

Рязань (180 км от Москвы), 4 часа назад

BMW 8 серии 840d xDrive II (G14/G15/G16) — 2018

6 544 000 ₽

от 62 150 ₽ / мес.

3.0 л / 320 л.с. / Дизель — Полный

Автомат — Синий

Купе

Отчёт ПроАвто

Саларьево, Филатов Луг, Москва, 1 час назад

Types of metrics

- Offline
  - Gathered once, used many times
  - Can show outdated results (no new items or users)
- Online
  - Gathered for particular setup, valid once
  - Limited quota for AB experiments
  - Show real situation (most of the times)

Offline metrics

girafe ai

01

Regression metrics

girafe ai

01

Root Mean Square Error

RMSE – was offered at the famous Netflix Prize competition

$$RMSE = \sqrt{\sum_{i=1}^{N} (score_i - rating_i)^2}$$

RMSE: disadvantages (for recommendation systems)

RMSE evaluates the quality of predicted ratings for all objects that users have interacted with, while the goal of most recommendation systems is to find the top N most relevant objects. Moreover, rmse can greatly over/under-tell, which is very bad.

It follows from this: ranking metrics used in information search are well suited for evaluating recommendation systems

Precision@k

girafe
ai

02

Precision@k

The proportion of relevant recommendations among the top k recommended

$$P@k = \frac{the\ number\ of\ relevant\ documents\ in\ top\ k}{k}$$

The key drawback is that it does not take into account the positions of documents

Precision@k

Prediction	Truth
0.90	?
0.88	1
0.78	?
0.67	0
0.66	1

Precision@3 = ?

Precision@k

Not all ratings are known!

Only known ratings should be taken into account when sorting!

Prediction	Truth
0.90	?
0.88	1
0.78	?
0.67	0
0.66	1

Precision@3 = 0.66

AveragePrecision@k

The sum of P@k for relevant documents divided by the number of relevant documents the user has.

$$AP@k = \frac{1}{n\_rel} \sum_{k=1}^{k} I \cdot P@k$$

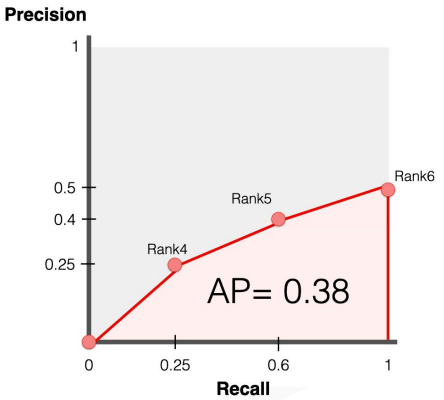
where I=1 if the document is relevant and 0 otherwise

Solves the problem of accounting for positions

		Precision @k	Recall @k
1		0	0
2		0	0
3		0	0
4		0.25	0.25
5		0.4	0.6
6		0.5	1

AP = 1.15/3 = 0.38

Not relevant Relevant



Mean Avg Precision@k (MAP@K)

Averaging AP@k over the entire test sample of size N

$$\mathrm{MAP@}k = \frac{1}{N}\sum_{i=1}^{N} AP@k_i$$

1. Binary relevance
2. Unknown coeffs for positions

Discounted Cumulative Gain

girafe
ai

03

Background

1. The most important documents in the top issue
2. Confusing the positions of the 90th and 100th aitems is not the same as confusing the 1st and 10th

How to ensure that the quality of ranking in the top of the output has a greater impact on the value of the metric?

Discount according to the position of the document!

DCG

DCG is the accumulated amount of document relevancy discounted by the logarithm of the document position in the output. Not normalized.

$$DCG_k = \sum_{i=1}^{k} \frac{rel_i}{log_2(i+1)}$$

Why discount on the logarithm?

$$DCG_k = \sum_{i=1}^{k} \frac{rel_i}{log_2(i+1)}$$

The logarithm mathematically expresses the premises of the formula:

$$\log_2 1 = 0, \log_2 10 = 3.32$$

$$\log_2 100 = 6.64, \log_2 110 = 6.78$$

Normalization

Usually we want metric to vary between 0 and 1, but this is not how DCG works.

Let’s normalize it using the highest value possible for a given items

$$NDCG_k = \frac{DCG_k}{IDCG_k}$$

NDCG: Example

Position in the issue	Ground Truth		Recommendations	
	Order of documents	Rating	Order of documents	Rating
1	d4	2	d3	2
2	d3	2	d2	1
3	d2	1	d4	2
4	d1	0	d1	0
DCG	**3.76**		**3.26**	
NDCG	**1.0**		**0.86**	

[Variants of calculations in Catboost]

$$DCG_{GT} = 1 + \frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} = 3.76$$

$$DCG_{rec} = 1 + \frac{1}{\log_2 2} + \frac{2}{\log_2 3} + \frac{0}{\log_2 4} = 3.26$$

$$NDCG = \frac{DCG_{rec}}{DCG_{GT}} = 0.86$$

Mean Reciprocal Rank

girafe ai

05

Mean Reciprocal Rank (MRR)

RR – reverse rank of the first relevant document

$$RR = \frac{1}{rel\_1st\_pos}, \qquad MRR = \frac{1}{N}\sum_{i=1}^{N} RR$$

Disadvantages: takes into account only the first relevant document, ignoring the subsequent ones

ROC AUC

Also applicable for ranking

Special properties

girafe
ai

06

Special properties

- Completeness of documents (coverage)
- Novelty (novelty)
- Diversity / personalization
- Wow effect (serendipity)

Completeness (Coverage)

Share of recommended objects I_P among all objects I

$$Coverage = \frac{|I_P|}{|I|}$$

Novelty / diversity

The key idea is that the less popular an object is, the more likely it is that it will be new to the user.

For each recommended object $i \in R$, we calculate the probability that a random user will have it $\mathrm{P}(i) = \frac{m_i}{N}$, , where m_i is how many people were shown the i-th object, and N is the total number of users.

This metric is sometimes called MSI (mean self-information)

$$Novelty_{user} = \frac{1}{R}\sum_{i \in R} -\log(P(i))$$

Diversity

						
Born This Way	*Pink Friday*	*Dangerously in Love*	*Born This Way – The Remix*	*Femme Fatale*	*Can't be Tamed*	*Teenage Dream*
Lady Gaga	Nicki Minaj	Beyoncé	Lady Gaga	Britney Spears	Miley Cyrus	Katy Perry

Diversity

Intra List Similarity (ILS) – we analyze the similarity of embeddings of items within the framework of recommendations.

We minimize the metric to achieve diversity.

$$ILS_{user} = \frac{1}{R} \sum_{i \in R} \sum_{j \in R} sim(i, j)$$

Diversity

Aggregate Diversity is the cumulative number of unique R_u objects that the model recommends to all U users.

$$Diversity_{agg} = \left| \bigcup_{u \in U} R_u \right|$$

Serendipity – creating a WOW effect

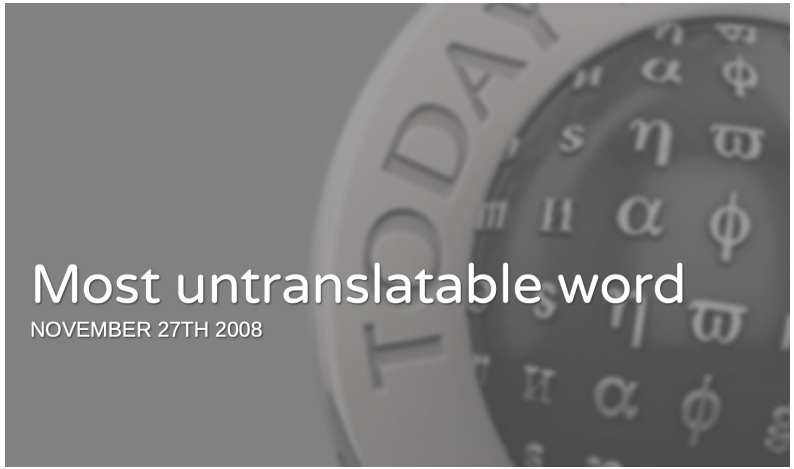
From the point of view of the recommendation system, it is the ability to recommend objects that are not only relevant to the user, but also differ significantly from what objects the user interacted with in the past.

Serendipity is a rather subjective property and it is difficult to formalize it, moreover, such recommendations are rare, which complicates their understanding and the possibility of measurement. There is no consensus on which metric to measure Serendipity

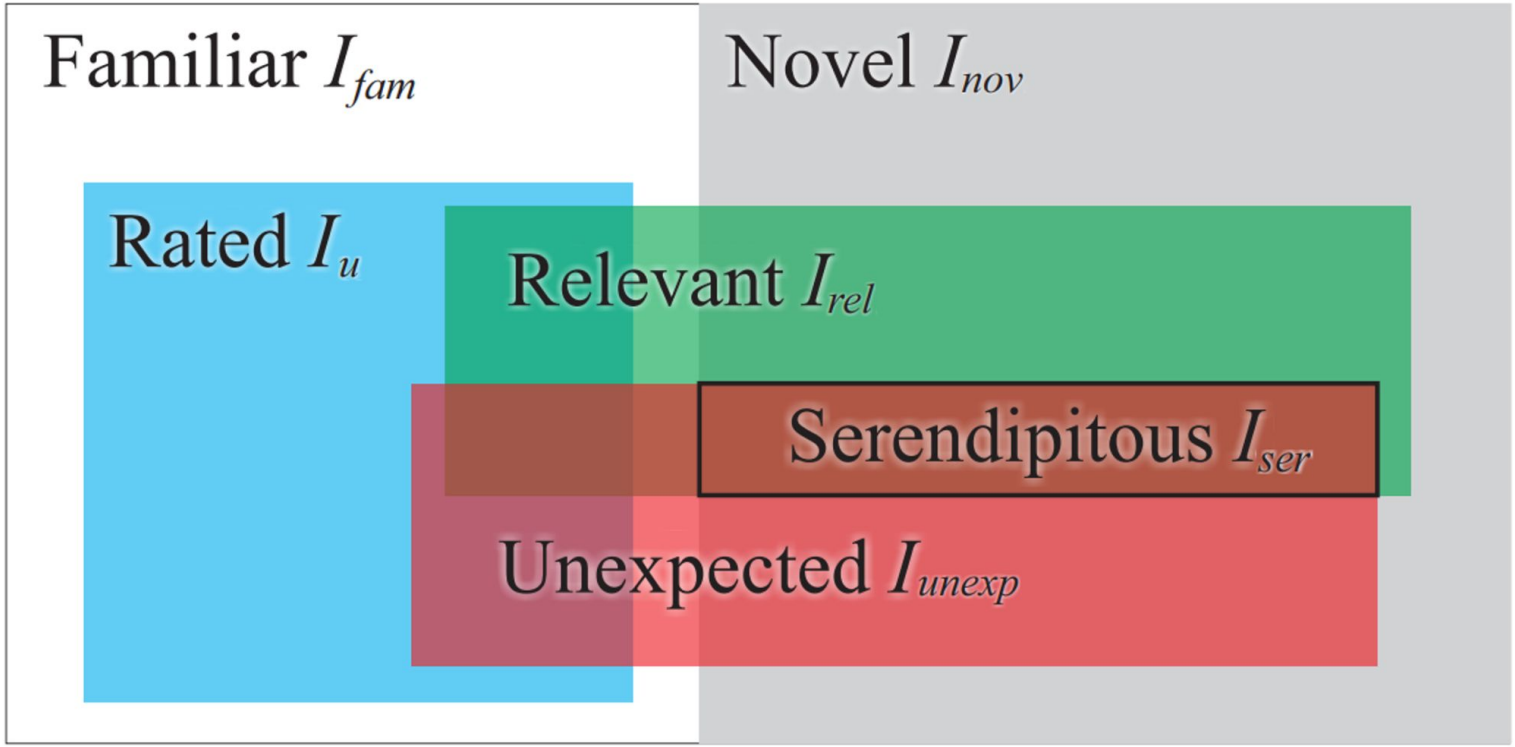
Serendipity - intuitive foresight

Senedipity is a term that entered the top10 words in 2008 with no explicit translation.

It is often translated into Russian as intuitive foresight



Serendipity



Serendipity

Let R_u be a list of recommendations for the user, $Pr_u(i)$ - prediction of the model, for each item from the list, and $Prim_u(i)$ - the prediction of a primitive model, and rel is the known relevance of the item for the user, then Serendipity is calculated as follows:

$$Serendipity_u = \sum_{i \in R_u} \max(Pr_u(i) - Prim_u(i), 0) \cdot rel_u(i)$$

To simplify the formula, the primitive model can be replaced by the predicate of the original model for a random user or by the average predicate of the model for all users

Online metrics

girafe ai

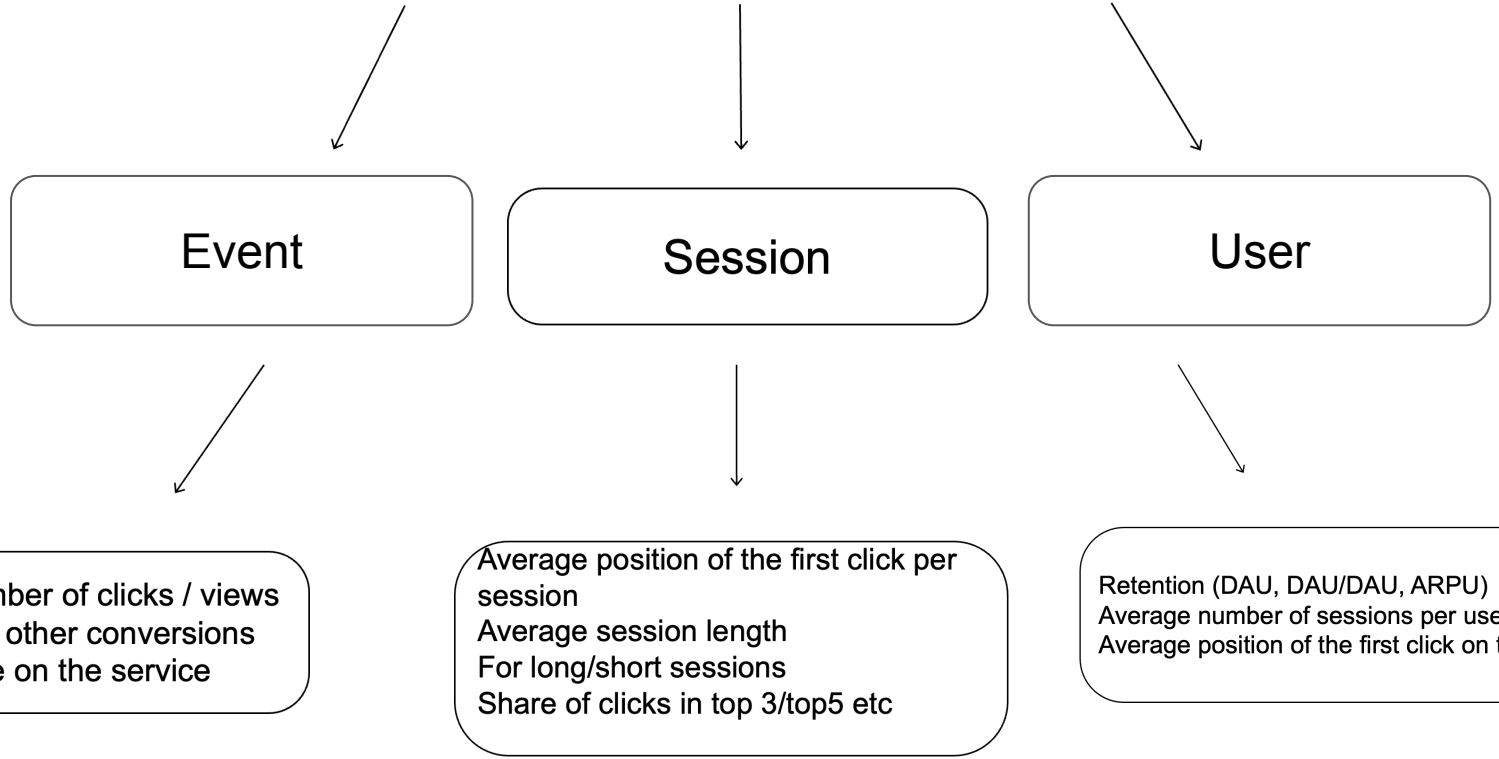
02

Online metrics

- Depend on the product
- Depend on business goals
- Influence the development of the product
- They are calculated on real users (most often in ab tests)
- and are most often the truth of last resort for making a decision about rolling out the model into production

It is critically important to choose the right "instruments" to assess the quality of recommendations

Online metrics



Product metrics: details

It is best to use custom metrics, because most often when we make a change in a product, we want to influence the user's perception of our product.

Event metrics are very dangerous. It is easy to make a mistake in interpreting the result if the metrics are implemented incorrectly. If you are faced with the task of designing an a/b testing platform – be careful

Event/session metrics are strongly influenced by robots / spam /fraud

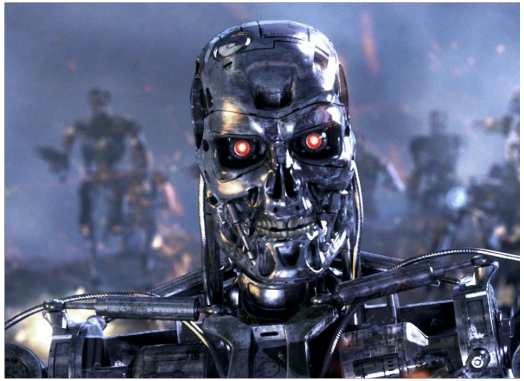
Product metrics: details

Example

Let our metric be the number of calls for an ad.

Run the a/b test, divide the sample into 2 splits. We run different recommendation algorithms on splits.

We see that one model gives the growth of the metric. However, robots could have screwed up this metric and, perhaps, the result of the split is incorrect. What to do?



Product metrics: details

Example

Let our metric be the number of calls for an ad.

Run the a/b test, divide the sample into 2 splits. We run different recommendation algorithms on splits.

We see that one model gives the growth of the metric. However, robots could have screwed up this metric and, perhaps, the result of the split is incorrect. What to do?

It is necessary to beat users by N packages within the split, calculate the average for them and compare with another split.

It turns out that we compare N honest random variables with similar random variables in another split, Such a trick will minimize the influence of robots on the metric value.

Product metrics is all you need?

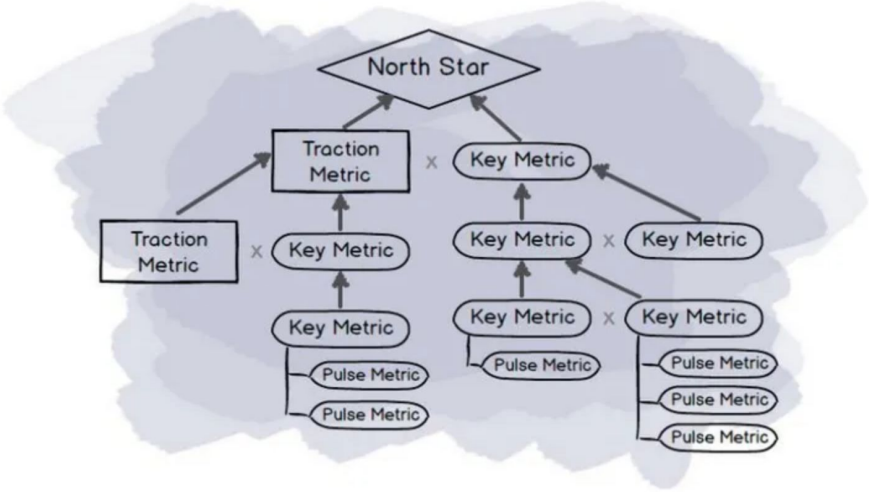
Focusing strictly on product metrics does not always lead to good results. It is important to measure the quality of service and the "happiness of users" so that a product based on a recommendation system has the potential for long-term growth.

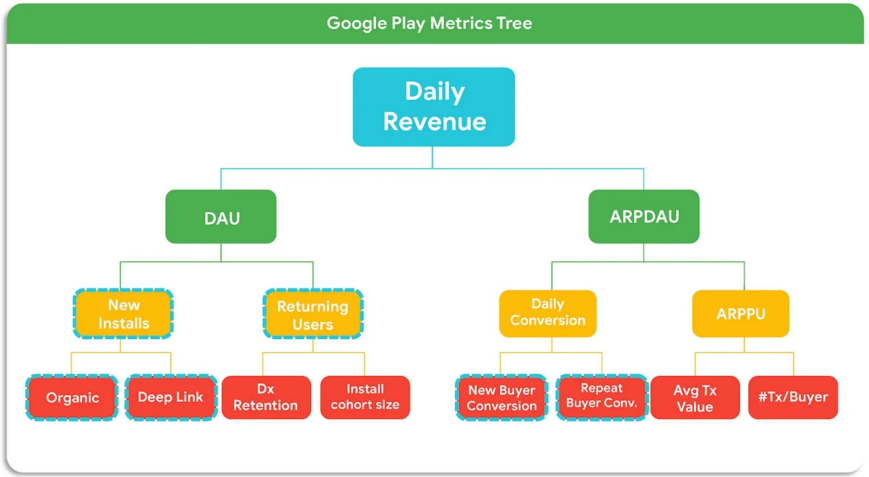
In other words, product metrics can approximate the quality of the service, but they do it poorly for long-term periods, you need to add other metrics to compensate for this myopia.

Metrics tree

All metrics for particular product (or problem) are organized in a tree structure expressing their priority for a final goal.

Usually Nord Star metric is placed into single root node of the tree to guide every solution.





Results

- Perfect metrics hardly exist
- The key metrics always come from the product
- It is important to remember that the growth of some metrics can cause the fall of others (the growth of novelty drops the accuracy, etc.)
- Examples of metrics implementation https://github.com/statisticianinstilettos/recmetrics/blob/master/recmetrics/metrics.py

Loss functions

girafe ai

03

Ranking problem statement

Дано N запросов (queries) Q = {q1, ..., qN}.

Каждый запрос состоит из K документов D = {d1, ..., dK}
и их меток релевантности L = {l1, ..., lK}

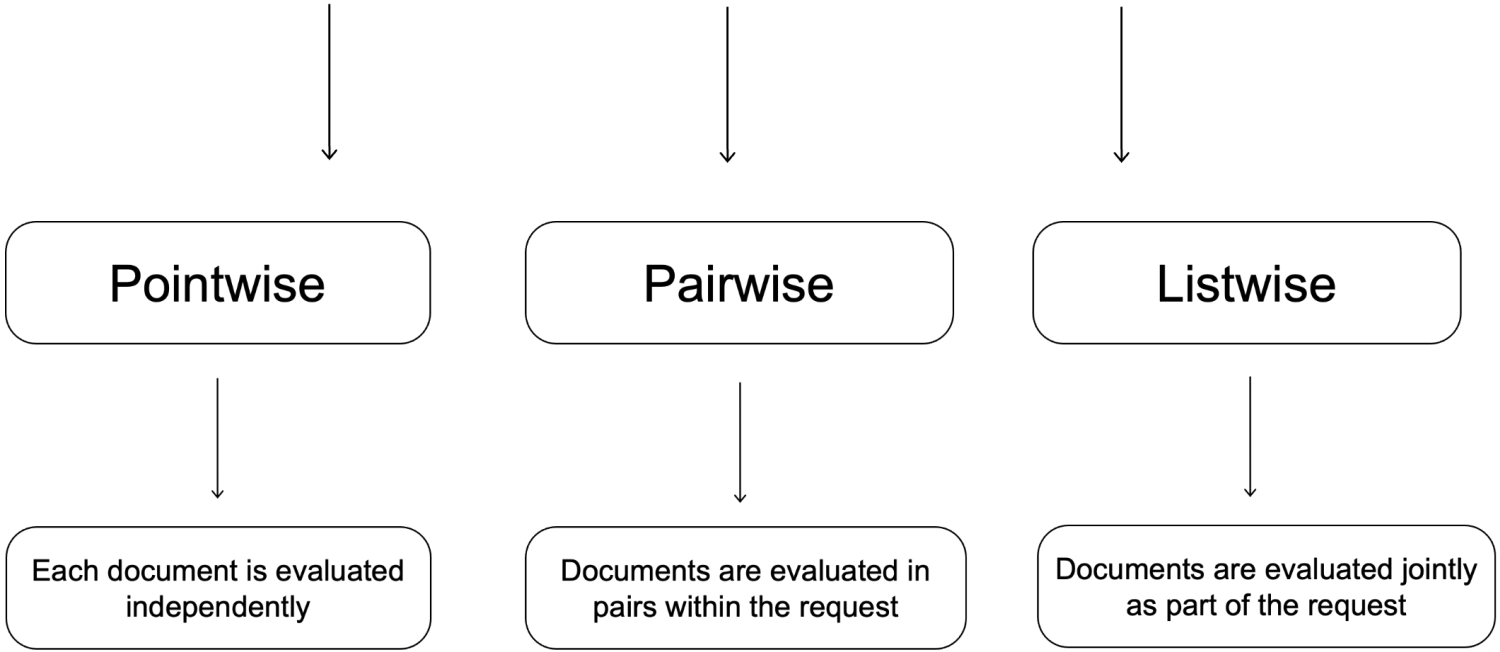
Каждый запрос qi описывается вектором признаков размерности E,
а документ di - размерности M.

Тогда цель обучения - это построить такую модель, которая наилучшим образом будет восстанавливать порядок документов в соответствии с истинными метками релевантности L.

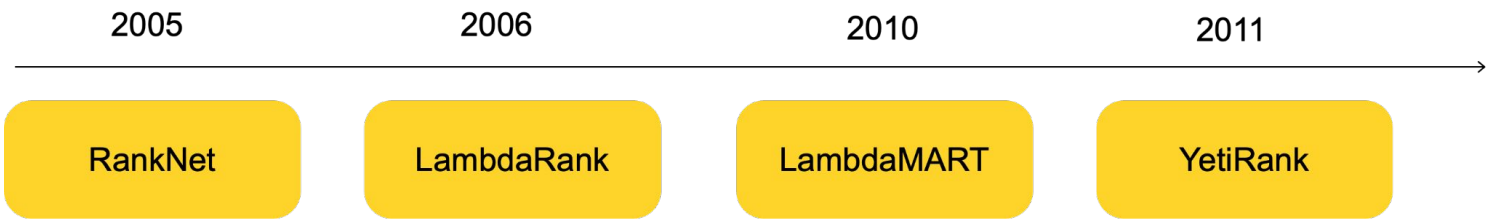
f(q, d1) = s1; f(q, d2) = s2; ...

sorted((sk, dk)) == sorted((lk, dk)). **sk != lk**

Approaches to solving the problem



Evolution of approaches



RankNet

girafe
ai

01

RankNet: Ranking training

$$P_{ij} \equiv P(U_i \rhd U_j) \equiv \frac{1}{1 + e^{-\sigma(s_i - s_j)}}$$

We train the model to predict the relevance of s for a pair of documents s_i, s_j. We substitute the difference of predicted relevancies into the sigmoid function (note that sigma in the denominator is a constant that regulates the scale - wtf MS research??) to predict the probability P_{ij} that the U_i document is more relevant than the U_j document

RankNet: loss function

$$C = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij})$$

$$\bar{P}_{ij} = \frac{1}{2}(1 + S_{ij})$$

Let S_{ij} be a target equal to 1 if the ith object is more relevant than the jth, -1 if the opposite is true and zero if the relevancy is equal.

RankNet: loss function and weight update

$$C = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij})$$

$$w_k \to w_k - \eta \frac{\partial C}{\partial w_k} = w_k - \eta \left(\frac{\partial C}{\partial s_i} \frac{\partial s_i}{\partial w_k} + \frac{\partial C}{\partial s_j} \frac{\partial s_j}{\partial w_k} \right)$$

RankNet: substituting values into the formula

$$C = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij})$$

$$C = \frac{1}{2}(1 - S_{ij})\sigma(s_i - s_j) + \log\left(1 + e^{-\sigma(s_i - s_j)}\right)$$

RankNet: the magic of differentiation

$$C = \frac{1}{2}(1 - S_{ij})\sigma(s_i - s_j) + \log\left(1 + e^{-\sigma(s_i - s_j)}\right)$$

Statement: the losses are symmetrical. The value of the derivative of the i-th aitem will be exactly equal to the value of the derivative of the j-th item with a minus sign. This property will be useful to us.

$$\frac{\partial C}{\partial s_i} = \sigma\left(\frac{1}{2}(1 - S_{ij}) - \frac{1}{1 + e^{\sigma(s_i - s_j)}}\right) = -\frac{\partial C}{\partial s_j}$$

RankNet: Factorization

$$\frac{\partial C}{\partial s_i} = \sigma\left(\frac{1}{2}(1 - S_{ij}) - \frac{1}{1 + e^{\sigma(s_i - s_j)}}\right) = -\frac{\partial C}{\partial s_j}$$

$$\frac{\partial C}{\partial w_k} = \frac{\partial C}{\partial s_i}\frac{\partial s_i}{\partial w_k} + \frac{\partial C}{\partial s_j}\frac{\partial s_j}{\partial w_k} = \sigma\left(\frac{1}{2}(1 - S_{ij}) - \frac{1}{1 + e^{\sigma(s_i - s_j)}}\right)\left(\frac{\partial s_i}{\partial w_k} - \frac{\partial s_j}{\partial w_k}\right)$$

$$= \lambda_{ij}\left(\frac{\partial s_i}{\partial w_k} - \frac{\partial s_j}{\partial w_k}\right)$$

We decompose the formula into the product of 2 components. This will speed up the training of the model and achieve almost linear complexity relative to the quadratic one before factorization.

Here we also introduce the concept of Lambda vectors λ_{ij}, which are gradients according to the predictions of the model for the pair i, j.

RankNet: due to what acceleration of calculations?

$$\delta w_k = -\eta \sum_{\{i,j\} \in I} \left(\lambda_{ij} \frac{\partial s_i}{\partial w_k} - \lambda_{ij} \frac{\partial s_j}{\partial w_k} \right) \equiv -\eta \sum_i \lambda_i \frac{\partial s_i}{\partial w_k}$$

The lambda gradient vector λ_i for document i is aggregated over all pairs (i,j) that can be formed in this query with the participation of the i-th document (where the relevance for the i-th is higher)

$$\lambda_i = \sum_{j:\{i,j\} \in I} \lambda_{ij} - \sum_{j:\{j,i\} \in I} \lambda_{ij}$$

Weights are now updated once per request, whereas previously they were updated for each pair of documents in the request. **Computational complexity is now linear in the number of documents inside the query, whereas it used to be quadratic.**

RankNet: the physical meaning of lambda vectors

Lambda gradient vector is a vector calculated for each document. The direction of the vector indicates whether the document needs to be moved up or down in relevance in order to improve the ranking.
The length of the vector is how much to move.

Original technical report:
https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf

LambdaRank

girafe ai

02

LambdaRank: optimize wisely

What are the disadvantages of RankNet? Let's take an example:

Suppose the model works almost perfectly, but it made 2 mistakes on the training sample for a given search result consisting of 99 documents:

1. Confuses the ranks of the 1st and 99th documents
2. Confuses the ranks of 97 and 98 documents

Which of these mistakes is more important to correct?
First

Is the price of these errors the same in terms of the loss function?
Yes, although it is obvious that their price is different.

Why is that?
Because RankNet optimizes the number of pairwise inversions in the ranked documents. Regardless of their positions in the output.

What to do?
Optimize directly some good ranking metric... for example, NDCG

LambdaRank: the key idea

LambdaRank uses the same lambda gradients as RankNet, only multiplies them by changing the DCG metric when rearranging the i-th and j-th document in places. The authors proved that in this case **NDCG is optimized directly!**

$$\lambda_{ij} = \frac{\partial C(s_i - s_j)}{\partial s_i} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} |\Delta_{NDCG}|$$

Moreover, it is true that it is possible to directly optimize an arbitrary ranking metric, be it MAP, MR, etc.

LambdaRank: again about lambda vectors

The gradients of RankNet are highlighted in black, LambdaRank is highlighted in red.

LambdaRank divides documents so as to maximize ndcg, while the gradients of the RankNet grow so as to minimize the number of pairwise ranking errors

LambdaMART

girafe
ai

03

LambdaMART: adaptation to boosting

LambdaMART = LambdaRank + MART

MART (Multiple Additive Regression Trees) – a family of boosting algorithms that can solve any class of problems (regression, classification, ranking). MART models the same lambda gradients, so it's easy to adapt it for ranking training.

LambdaRank and LambdaMART

- LambdaRank updates all the weights after each query is examined
- LambdaMART updates only a few parameters at a time

The decisions (splits at the nodes) in LambdaMART are computed using all the data that falls to that node, and so (namely, the split values for the current leaf nodes), but using all the data (since every x_i lands in some leaf). This means in particular that LambdaMART is able to choose splits and leaf values that may decrease the utility for some queries, as long as the overall utility increases

Article: How LambdaMART works

YetiRank

girafe ai

04

YetiRank

Intuition:

1) Not all document pairs are equally useful. You need to weigh it.
2) The most important pairs are those that meet in the top of the rating
3) The model's confidence is important that the i-th document is relevant to the j-th. Regularization for uncertain prediction.

$$\mathbb{L} = -\sum_{(i,j)} w_{ij} \log \frac{e^{x_i}}{e^{x_i} + e^{x_j}},$$

where x_i, x_j are the relevance predictions for the given documents, w_{ij} is the weight for the pair i, j

More info in paper https://proceedings.mlr.press/v14/gulin11a.html

MART is used as a boosting algorithm

Practical notes

- https://arxiv.org/pdf/2204.01500

Great lecture on ranking from Sergei Nikolenko [ru]

https://www.youtube.com/watch?v=9tNZTPPxKI8

Overview of GB implementations

- XGBoost (LambdaMART, …)
  https://xgboost.readthedocs.io/en/latest/parameter.html
- CatBoost (YetiRank, …)
  https://catboost.ai/en/docs/concepts/loss-functions-ranking#pairwise-objectives-and-metrics
  - Tutorial
    https://github.com/catboost/catboost/blob/master/catboost/tutorials/ranking/ranking_tutorial.ipynb
- LightGBM (LambdaRank, …)
  https://lightgbm.readthedocs.io/en/latest/Parameters.html

Overview of NN implementations

- Casual intro https://medium.com/@mandeep0405/learning-to-rank-ranknet-simplified-5d7f7334133d
- PT ranking https://wildltr.github.io/ptranking/
- allRank (now abandoned) https://github.com/allegro/allRank

End to end system

- https://github.com/metarank/metarank

Useful materials

- Theory of Learning-to-Rank [ru] https://www.youtube.com/watch?v=CVnxexNVUbk
- Practical Learning-to-Rank https://www.youtube.com/watch?v=oXfFqAKf4Ac
-

Papers

1) C.J.C.Burges,T.Shaked,E.Renshaw,A.Lazier,M.Deeds,N.HamiltonandG.Hullender. Learning to Rank using Gradient Descent. Proceedings of the Twenty Second International Conference on Machine Learning, 2005

2) C.J.C. Burges, R. Ragno and Q.V. Le. Learning to Rank with Non-Smooth Cost Functions. Advances in Neural Information Processing Systems, 2006

3) J.H. Friedman. Greedy function approximation: A gradient boosting machine. Technical Report, IMS Reitz Lecture, Stanford, 1999; see also Annals of Statistics, 2001.

4) C.J.C. Burges. From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82

5) Gulin, A., Kuralenok, I., Pavlov, D.: Winning the transfer learning track of Yahoo!'s learning to rank challenge with YetiRank. JMLR: Workshop and Conference Proceedings 14 (2011) 63–76

Formats mixing problem

girafe
ai

05

Blender concept

Рассказать про устройство блендера в Дзене и почему обучаемые блендеры очень сложны в эксплуатации

Thanks for attention!

Questions?